



universidad
de león



Departamento de Ingenierías
Mecánica, Informática y Aeroespacial

MÁSTER UNIVERSITARIO EN ROBÓTICA E INTELIGENCIA ARTIFICIAL

Visión por Computador

CLASIFICACIÓN DE TEXTURAS

Carlos González Carballo.

Repositorio Github: [Carlos.gcg00/Vision por Computador](https://github.com/Carlos.gcg00/Vision_por_Computador)

Índice general

Índice de figuras	II
Índice de tablas	IV
1. Introducción	1
2. Análisis del dataset.	2
3. Clasificación del dataset empleando descriptores	3
3.1. Descriptores Globales	4
3.1.1. Histogram of Oriented Gradients - HOG	4
3.2. Gray Level Co-occurrence Matrix - GLCM	8
3.3. Local Binary Pattern - LBP	10
3.4. Descriptores Locales	13
3.4.1. Harris Detector	13
3.4.2. Canny Edge Detector	16
3.4.3. Scale Invariant Feature Transform (SIFT)	18
3.4.4. Bag of Visual Words - BoVW	21
3.4.5. Vector of Locally Aggregated Descriptors - VLAD	22
4. Clasificación del dataset empleando Redes Neuronales	26
4.1. Fully Connected	27
4.1.1. Clasificación de texturas	27
4.2. CNN	29
4.2.1. Clasificación de texturas	29
5. Conclusiones	32
Bibliografía	34

Índice de figuras

2.1. 6 Muestras del dataset	2
3.1. Histogram of Oriented Gradients	6
3.2. Matriz de confusión + PCA, SGD(HOG)	7
3.3. Matriz de confusión + PCA, KNN(HOG)	8
3.4. GLCM	9
3.5. Matriz de confusión, SGD(GLCM)	10
3.6. Matriz de confusión, KNN(GLCM)	10
3.7. LBP	11
3.8. LBP con <code>skimage</code>	12
3.9. Matriz de confusión, SGD(LBP)	12
3.10. Matriz de confusión, KNN(LBP)	13
3.11. Harris	14
3.12. Matriz de confusión, SGD(Harris)	15
3.13. Matriz de confusión, KNN(Harris)	16
3.14. Canny	17
3.15. Matriz de confusión, SGD(Canny)	17
3.16. Matriz de confusión, KNN(Canny)	18
3.17. Matriz de confusión, SGD(SIFT)	20
3.18. Matriz de confusión, KNN(SIFT)	21
3.19. BoVW	22
3.20. Matriz de confusión, SGD(SIFT + BoVW)	23
3.21. Matriz de confusión, KNN(SIFT + BoVW)	23
3.22. Matriz de confusión, SGD(VLAD)	24
3.23. Matriz de confusión, KNN(VLAD)	25
4.1. FC	27
4.2. Arquitectura de FC	28
4.3. Evolución de <i>accuracy</i> y <i>loss</i> con arquitectura FC	28

4.4. Matriz de confusión del modelo FC	29
4.5. Arquitectura de CNN	30
4.6. Evolución de <i>accuracy</i> y <i>loss</i> con arquitectura CNN	30
4.7. Matriz de confusión del modelo CNN	31

Índice de tablas

3.1. Resultados de clasificación SGD(HOG)	7
3.2. Resultados de clasificación SGD(GLCM)	9
3.3. Resultados de clasificación SGD(LBP)	12
3.4. Resultados de clasificación SGD(Harris)	15
3.5. Resultados de clasificación SGD(Canny)	17
3.6. Resultados de clasificación SGD(SIFT)	20
3.7. Resultados de clasificación SGD(SIFT + BoVW)	22
3.8. Resultados de clasificación SGD(VLAD)	24
4.1. Resultados de clasificación FC	28
4.2. Resultados de clasificación CNN	30

Capítulo 1

Introducción

Este proyecto trata de resolver la segunda práctica propuesta en la asignatura de Visión por Computadora del máster MURIA. En esta práctica se pretende realizar la clasificación de un dataset de imágenes de texturas.

Para la clasificación de texturas, habrá dos metodologías a analizar. La primera es emplear descriptores de imágenes tradicionales, descriptores locales y globales que proporcionarán características de cada una de las imágenes; posteriormente, se entrenará un modelo de Machine Learning para resolver la tarea de clasificación, pero este modelo empleará estas características como entrada. La segunda manera es emplear arquitecturas de Redes Neuronales para resolver la tarea.

Este documento se divide en 5 capítulos: Capítulo 1 que es el actual, la Introducción, Capítulo 2 consiste en la presentación y análisis del dataset, Capítulo 3 donde se presentan y aplican los descriptores locales y globales y se aplica un modelo de ML para su clasificación, 4 análisis de arquitecturas de Redes Neuronales para la clasificación de texturas y por último, Capítulo 5 conclusiones.

Capítulo 2

Análisis del dataset.

El dataset que se va a emplear es el *Kyleberg Texture Dataset* [1], que cuenta con un total de 240 imágenes con 6 texturas diferentes. El dataset está balanceado, por tanto, hay 40 imágenes por textura. El tamaño de las imágenes es el mismo para todas ellas, siendo de 576x576 píxeles. La siguiente figura ilustra un ejemplo de cada una de las texturas del dataset 2.1.

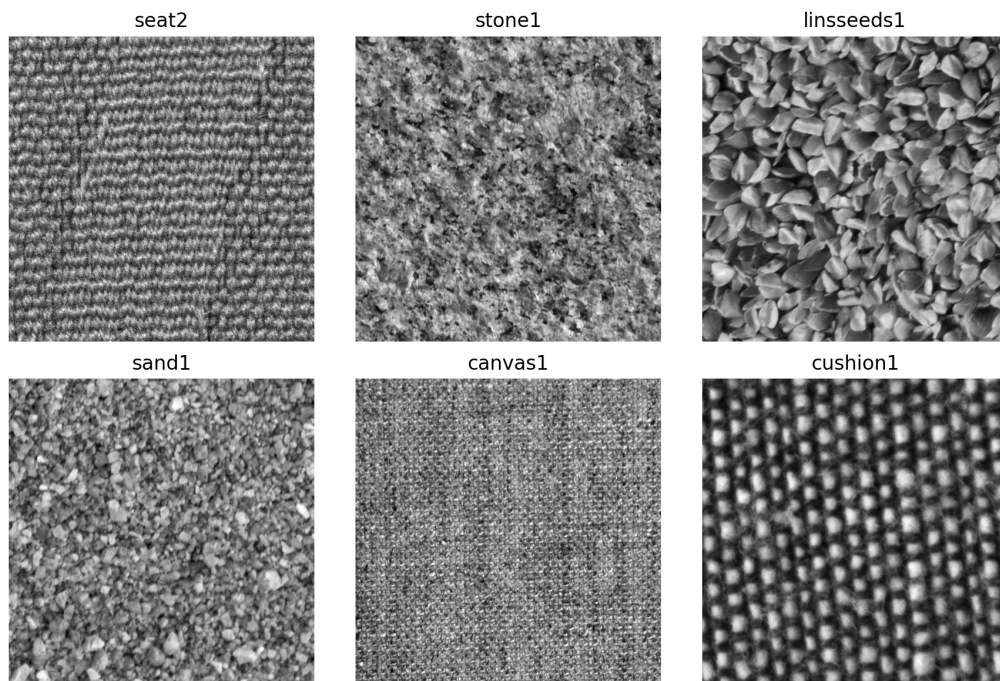


Figura 2.1: 6 Muestras del dataset

Como podemos observar, el dataset proporcionado se encuentra en escala de grises; por tanto, solo contará con un único canal.

Capítulo 3

Clasificación del dataset empleando descriptores

Si se emplean directamente los píxeles de la imagen como entrada para el modelo de clasificación, se obtiene un vector de entrada de alta dimensión (no deseable). Además, los píxeles son muy sensibles a cambios de escala, rotaciones, iluminación y ruido.

Al utilizar descriptores, se mitigan los problemas anteriores, ya que estos extraen características relevantes de las imágenes que, de forma habitual, se emplean como entrada para un modelo de *Machine Learning* (ML), permitiendo así clasificar las imágenes de manera más robusta.

Existen diferentes maneras de realizar la extracción de características:

- **Descriptores globales:** extraen características genéricas de la imagen a lo largo de toda su extensión.
- **Descriptores locales:** analizan la imagen en pequeñas regiones. Algunos descriptores locales utilizados en esta situación, como SIFT, dan lugar a vectores de dimensión uniforme.

Las características extraídas en cada uno de los casos se utilizarán como entrada en dos modelos de *Machine Learning*: ***Stochastic Gradient Descent*** (SGD) y ***KNN***, con el objetivo de comparar el comportamiento de ambos. Se utilizarán las siguientes métricas de evaluación: ***accuracy***, ***precision***, ***recall*** y ***f1-score***. Además, se presentarán dos tipos de gráficos: la matriz de confusión sobre los datos de evaluación y un *scatter plot* que mostrará las tres componentes principales tras

aplicar PCA a los datos. La nube de puntos se coloreará según la clase, marcando con una O las predicciones correctas y con una X los errores.

Aunque a veces este tipo de representación no permite extraer conclusiones claras, proporciona una primera visión sobre cómo se distribuyen los datos en el espacio de características. El *scatter plot* presentado enfrenta PC1–PC2, PC2–PC3 y PC1–PC3; es posible que en una de estas combinaciones se observen patrones que no se aprecian en las demás. Cabe destacar que esta representación es únicamente visual y no refleja directamente el rendimiento del modelo. La nube de puntos será la misma para los modelos entrenados con un mismo descriptor; lo que cambiará serán las predicciones (aciertos y errores).

Nota: el sistema se ha preparado para poder combinar diferentes descriptores de manera sencilla. Sin embargo, finalmente no se han combinado, ya que se observaron buenos resultados utilizando cada descriptor por separado.

A priori, la configuración de los modelos será la misma para evaluar los diferentes descriptores, salvo que el rendimiento sea claramente insuficiente.

- **Configuración inicial de SGD:** se crea un *pipeline* con una estandarización previa de los datos, dado que este modelo es sensible a la escala. Además, se integra una penalización L2 con $\alpha = 0.0001$.
- **KNN:** la única configuración que se ajustará será el número de vecinos, que se irá modificando en cada caso.

Para el entrenamiento de los modelos se ha decidido dividir el dataset en los siguientes porcentajes: 70 % para entrenamiento y 30 % para validación. Como se sabe que en el dataset completo hay el mismo número de muestras para cada clase, la división se realiza de forma balanceada. Como resultado, se obtienen 168 imágenes para entrenamiento (28 imágenes por textura) y 72 imágenes para validación (12 imágenes por textura).

3.1. Descriptores Globales

3.1.1. Histogram of Oriented Gradients - HOG

HOG es un descriptor de características que utiliza la distribución (histograma) de las orientaciones de los gradientes. El uso de los gradientes es especialmente relevante, ya que en regiones con bordes o esquinas la magnitud del gradiente suele ser elevada.

Nota: HOG se calcula por zonas de la imagen y, por tanto, es un descriptor local. Sin embargo, al concatenar todos los descriptores de las distintas celdas se obtiene un descriptor global de la imagen.

El cálculo de HOG se divide en cinco pasos:

1. **Preprocesamiento:** conversión de la imagen a escala de grises.
2. **Cálculo de gradientes:** se calculan los gradientes de la imagen en las direcciones x e y , G_x y G_y , y posteriormente se obtienen tanto la magnitud como la dirección del gradiente.

Para calcular los gradientes se utiliza la máscara de Sobel:

- Filtro horizontal:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

- Filtro vertical:

$$G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

El ángulo del gradiente se calcula como:

$$\theta = \arctan\left(\frac{G_y}{G_x}\right)$$

y la magnitud como:

$$M = \sqrt{G_x^2 + G_y^2}$$

3. **Cálculo de histogramas:** la imagen se divide en celdas de $M \times M$ píxeles (típicamente 8×8). Para cada celda se calcula un histograma de orientaciones de gradiente con 9 *bins*, que cubren el rango de 0 a 180 grados. De este modo, cada bin representa un intervalo de 20 grados (bin 0: [0,20), bin 1: [20,40), bin 2: [40,60), etc.).
4. **Normalización de bloques:** una vez calculados los histogramas de gradientes para cada celda, se agrupan las celdas en bloques y se normaliza el vector de características de cada bloque para reducir la sensibilidad a cambios de iluminación.

Un bloque se define como una matriz de $N \times N$ celdas, típicamente 2×2 . Si cada celda tiene 8×8 píxeles, un bloque ocupará 16×16 píxeles. Dado que

cada celda aporta un histograma de 9 *bins*, un bloque de 2×2 celdas genera, tras concatenar los histogramas de las 4 celdas, un vector de 36 *bins*. Este descriptor se normaliza habitualmente con la norma L2.

5. **Concatenación de descriptores:** el proceso anterior se repite para todas las ventanas de la imagen, y todos los vectores resultantes se concatenan para formar el descriptor final.

Dado que la imagen tiene un tamaño de 576x576 píxeles, utilizando bloques de 2×2 celdas, celdas de 8×8 píxeles y un *stride* de 8 píxeles, se obtendrá un total de:

$$\left(\frac{576 - 2 \cdot 8}{8} + 1, \frac{576 - 2 \cdot 8}{8} + 1 \right) = (71, 71)$$

Como cada bloque se describe con un vector de 36 *bins*, el descriptor final tendrá:

$$71 \cdot 71 \cdot 36 = 181,476$$

componentes.

La Figura 3.1 muestra una imagen de ejemplo y el resultado de aplicar HOG, donde se visualizan los vectores de gradiente (magnitud y dirección). Dado que se trata de una textura muy variable en regiones muy pequeñas, el patrón no es fácilmente interpretable mediante inspección visual.

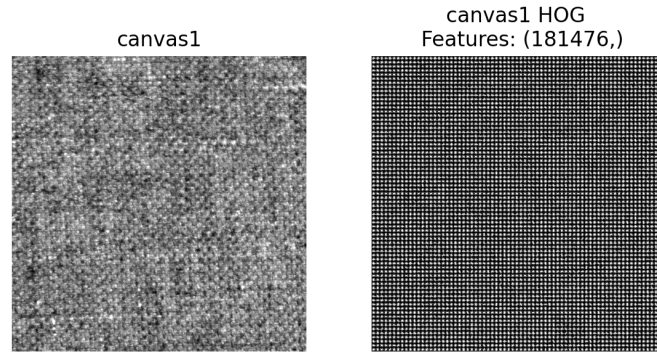


Figura 3.1: Histogram of Oriented Gradients

Clasificación de texturas

El primer paso fue definir el modelo de clasificación basado en SGD, empleando la función de pérdida *hinge* y una regularización con penalización L2 con $\alpha = 0.0001$. A continuación, se entrenó el modelo y se evaluó su rendimiento.

La Tabla 3.1 resume las métricas obtenidas tanto para SGD con la configuración descrita como para un KNN con 3 vecinos, e incluye también el número de parámetros utilizados por este descriptor. Las Figuras 3.2 y 3.3 muestran la matriz de confusión y el *scatter plot* para SGD y KNN, respectivamente. Se observa que, en este caso, SGD alcanza métricas razonablemente buenas, mientras que KNN presenta un rendimiento muy bajo. Hay que tener en cuenta que el descriptor HOG genera un vector de gran dimensión y KNN se basa en la distancia euclídea, lo que puede conducir a soluciones poco satisfactorias (los descriptores posteriores mejorarán este comportamiento).

En el caso de KNN, la matriz de confusión revela que el modelo tiende a predecir siempre la misma clase; esto sugiere que el algoritmo no está bien optimizado en este espacio y cae en un sesgo fuerte hacia una clase concreta (por ejemplo, “seat2”).

Tabla 3.1: Resultados de clasificación SGD(HOG)

Modelo	Descriptor	Num Params	Accuracy	Precision	Recall	F1-score
SGD	HOG	181476	0.71	0.75	0.71	0.71
KNN	HOG	181476	0.17	0.03	0.17	0.05

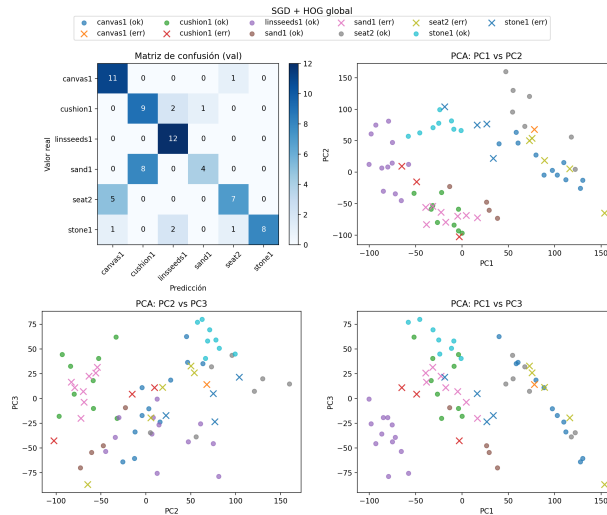


Figura 3.2: Matriz de confusión + PCA, SGD(HOG)

Nota: Para el entrenamiento del modelo SVC se ha empleado Google Colab, debido a las limitaciones de recursos del equipo local.

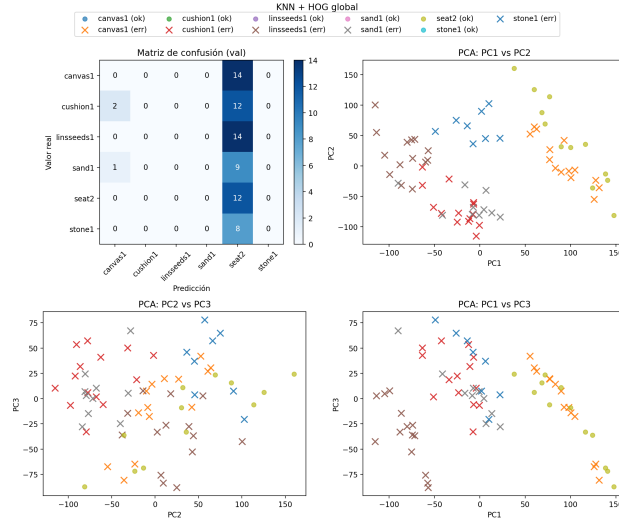


Figura 3.3: Matriz de confusión + PCA, KNN(HOG)

3.2. Gray Level Co-occurrence Matrix - GLCM

GLCM es un método estadístico utilizado para analizar la textura de las imágenes. Tiene en cuenta la información espacial y la frecuencia con la que aparecen pares de niveles de gris a una cierta distancia y en una determinada dirección.

En primer lugar, se debe fijar la distancia entre píxeles y la dirección. A continuación, se obtiene la lista de pares de píxeles para la distancia y dirección seleccionadas. Posteriormente, se calculan los valores únicos de estos pares y su frecuencia a lo largo de la imagen. Finalmente, se construye la matriz de co-ocurrencia y, a partir de ella, se extraen características estadísticas.

Entre las características más comunes se encuentran:

- **Contraste:** mide la diferencia de intensidad entre un píxel y sus vecinos en toda la imagen.
- **Disimilitud:** mide cuánto difieren los niveles de gris de los píxeles vecinos.
- **Correlación:** mide qué tan correlacionado está un píxel con sus vecinos.
- **Energía:** mide la uniformidad de la distribución de la textura.
- **Homogeneidad:** mide la similitud entre los valores de los píxeles vecinos.
- **ASM:** (*Angular Second Moment*) mide el orden y la uniformidad de la textura.

Este método se puede aplicar tanto a la imagen completa como a regiones de la misma. En principio, se analizará únicamente la GLCM de la imagen completa

sin segmentarla en regiones (el código está preparado para permitir también este análisis regional).

En este trabajo se ha estudiado la GLCM para una distancia 1 y para cuatro direcciones:

$$[0, \pi/4, \pi/2, 3\pi/4]$$

Para cada distancia y dirección se calculan las 6 características anteriores, obteniendo un vector de longitud 6. Al utilizar cuatro direcciones y concatenar los vectores resultantes, se obtiene un descriptor de longitud 24. La Figura 3.4 muestra la visualización de GLCM para una imagen de ejemplo, para distancia 1 en las cuatro direcciones.

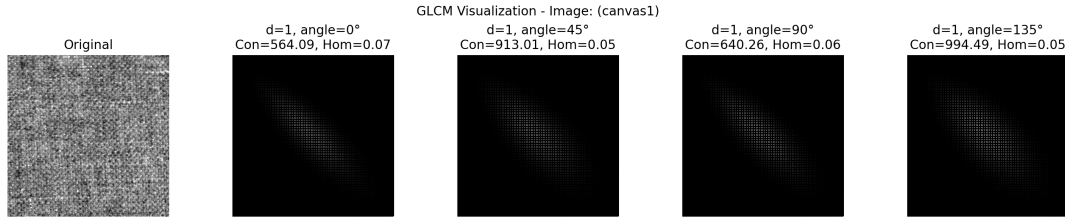


Figura 3.4: GLCM

Clasificación de texturas

La Tabla 3.2 resume las métricas obtenidas con GLCM como descriptor para los modelos SGD y KNN. Las Figuras 3.5 y 3.6 muestran las correspondientes matrices de confusión y las nubes de puntos tras aplicar PCA.

Se observa que, para este descriptor, la nube de puntos generada permite identificar los grupos de manera clara, lo que facilita la tarea de clasificación. En consecuencia, ambos modelos alcanzan la precisión máxima.

Tabla 3.2: Resultados de clasificación SGD(GLCM)

Modelo	Descriptor	Num Params	Accuracy	Precision	Recall	F1-score
SGD	GLCM	24	1.00	1.00	1.00	1.00
KNN	GLCM	24	1.00	1.00	1.00	1.00

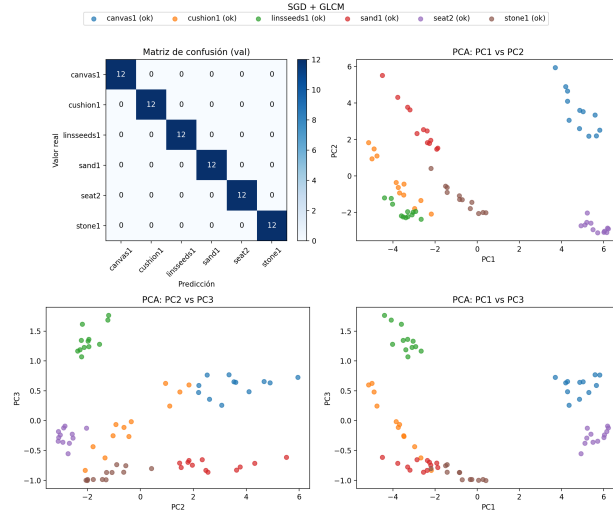


Figura 3.5: Matriz de confusión, SGD(GLCM)

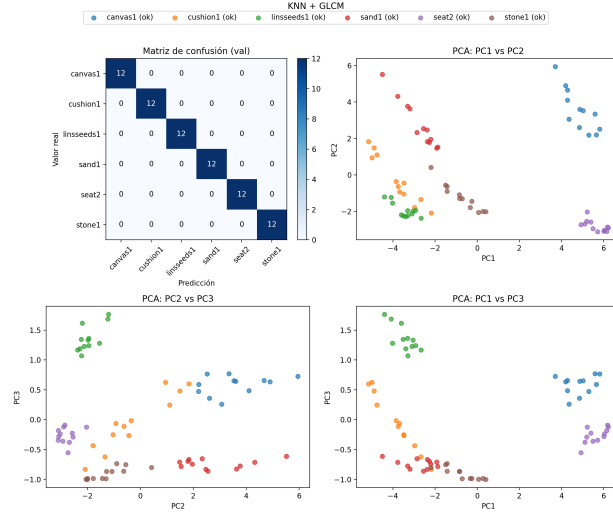


Figura 3.6: Matriz de confusión, KNN(GLCM)

3.3. Local Binary Pattern - LBP

LBP es un método ampliamente utilizado en visión por computador y, en particular, en problemas de análisis de texturas. Funciona codificando la información local de la imagen mediante la comparación de cada píxel con sus vecinos.

En esencia, LBP asigna un código binario a cada píxel en función de la comparación con la intensidad de sus vecinos.

El procedimiento es el siguiente:

1. Conversión de la imagen a escala de grises.

2. Para cada píxel, se obtienen sus vecinos. Considerando un vecindario de 3×3 , el píxel central es el objetivo y tendrá 8 vecinos.
3. Se compara la intensidad del píxel objetivo con la de cada vecino. Si la intensidad del vecino I es mayor que la del píxel objetivo, se asigna un 1; en caso contrario, se asigna un 0.
4. Se concatenan los 8 valores binarios en sentido horario, formando un código de 8 bits. (En la implementación desarrollada se permite elegir si la concatenación empieza en la esquina superior izquierda o en la superior derecha).
5. El código binario se convierte a decimal y se asigna al píxel correspondiente en la imagen de salida.
6. El proceso se repite para todos los píxeles de la imagen.
7. Se construye un histograma de los valores obtenidos. Como se tienen códigos de 8 bits, los valores posibles van de 0 a 255. Para mejorar la representación, se agrupan los valores en *bins*, dependiendo del parámetro `N_bins`, de forma que se divide el rango en 2^{N_bins} *bins*.

El tamaño final del vector descriptor depende del número de *bins*. Dado el rango 0–255 y usando 2^{N_bins} *bins*, el número total de componentes será $256/2^{N_bins}$. En este trabajo, se selecciona `N_bins` = 3, por lo que se obtiene un vector de tamaño 32.

La Figura 3.7 ilustra el resultado de aplicar LBP a una imagen original. La imagen central corresponde a la codificación tras convertir los valores binarios a decimales, y la de la derecha muestra el histograma asociado.

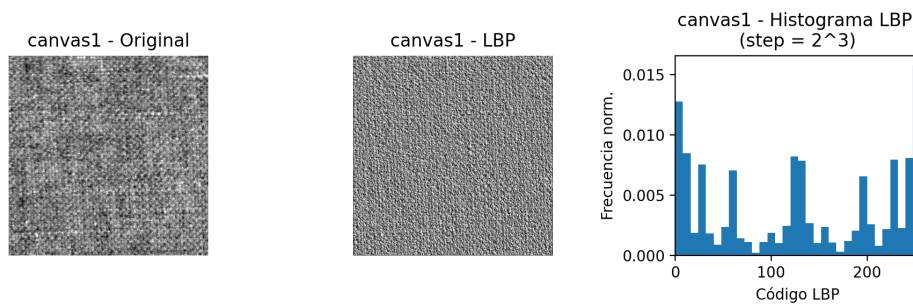


Figura 3.7: LBP

La imagen anterior se obtuvo con la primera implementación desarrollada, poco optimizada y basada en muchos bucles, lo que la hacía lenta. Finalmente, se integró también la función disponible en la librería de Python *skimage*, que incluye

una implementación eficiente del algoritmo. La Figura 3.8 muestra el resultado en este caso; se parece mucho a la implementación inicial, aunque existen pequeñas diferencias, probablemente debidas a algún detalle en el desarrollo propio.

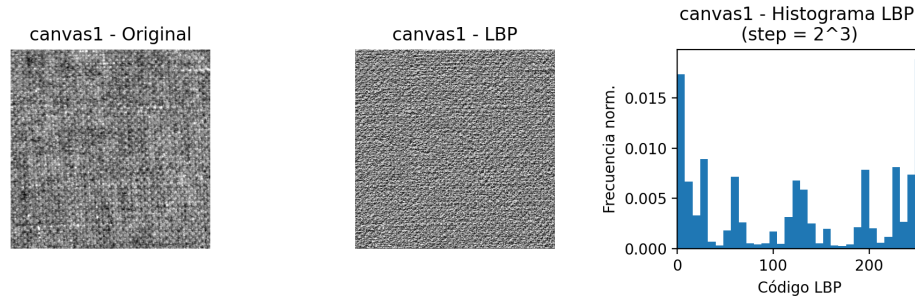


Figura 3.8: LBP con skimage

Clasificación de texturas

Al igual que en el caso de GLCM, aunque aquí se emplean 8 parámetros más, las métricas obtenidas son excelentes (Tabla 3.3, Figuras 3.9 y 3.10).

Tabla 3.3: Resultados de clasificación SGD(LBP)

Modelo	Descriptor	Num Params	Accuracy	Precision	Recall	F1-score
SGD	LBP	32	0.99	0.99	0.99	0.99
KNN	LBP	32	1.00	1.00	1.00	1.00

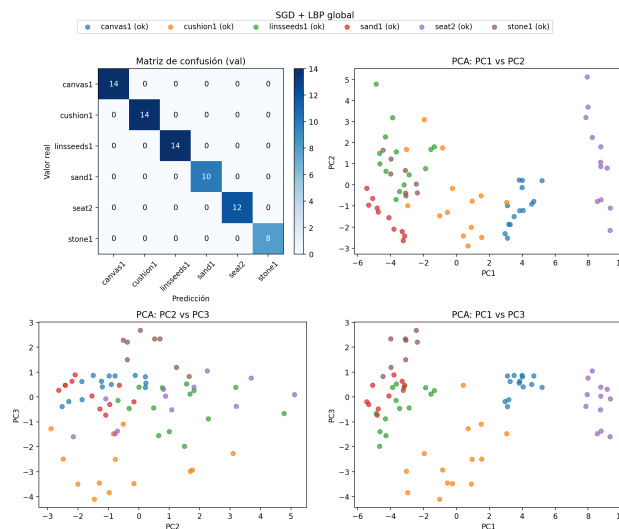


Figura 3.9: Matriz de confusión, SGD(LBP)

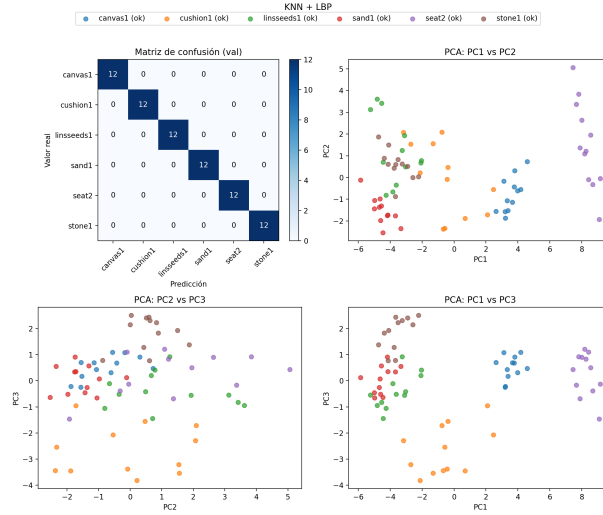


Figura 3.10: Matriz de confusión, KNN(LBP)

3.4. Descriptores Locales

3.4.1. Harris Detector

El detector de Harris es un descriptor local que se encarga de detectar esquinas en la imagen. En este proceso, los gradientes juegan un papel fundamental, ya que una gran variación en la intensidad suele indicar la presencia de una esquina.

Los pasos que sigue este extractor de características son:

1. **Cálculo de gradientes:** para cada píxel en escala de grises se calculan los gradientes en x e y , G_x y G_y .
2. **Construcción del tensor de estructura:** para cada píxel se calculan los elementos del tensor de estructura aplicando una función de suavizado gaussiano sobre los gradientes al cuadrado:

$$M = \begin{bmatrix} G_{xx} & G_{xy} \\ G_{yx} & G_{yy} \end{bmatrix}$$

3. **Cálculo de la respuesta de esquina:** utilizando la matriz M , se calcula el determinante y la traza para obtener el valor de Harris:

$$R = \det(M) - k(\text{trace}(M))^2$$

4. **Umbralización:** se aplica un umbral para decidir qué píxeles corresponden a esquinas:

$$R > \text{threshold} \implies \text{esquina}$$

La función de respuesta de esquina R depende de los autovalores de la matriz M , denotados por λ_1 y λ_2 . Se sabe que:

- Si $\lambda_1 \approx \lambda_2 \approx 0$, el píxel pertenece a una región plana (*flat region*).
- Si $\lambda_1 \gg 0$ y $\lambda_2 \approx 0$ (o viceversa), el píxel pertenece a un borde (*edge*).
- Si $\lambda_1 \gg 0$ y $\lambda_2 \gg 0$, el píxel pertenece a una esquina (*corner*).

Harris es, estrictamente hablando, un detector de esquinas, no un extractor de características completo, ya que su objetivo principal es localizar puntos de interés y no describirlos. En teoría, se podría construir un descriptor (poco eficiente) utilizando la respuesta de Harris: primero detectar las esquinas, después extraer descriptores locales alrededor de ellas (de forma análoga a SIFT) y, por último, aplicar técnicas de agregación como BoVW, VLAD u otras similares.

En este trabajo se propone una alternativa más sencilla: en lugar de construir descriptores locales, se extraen estadísticos globales que resumen la respuesta de Harris en toda la imagen. Concretamente, se calcula el porcentaje de píxeles clasificados como esquinas, el valor máximo de la respuesta de Harris, así como la media y la desviación estándar de dicha respuesta. Este proceso convierte la información de Harris en un descriptor global.

Como resultado, el vector descriptor final tiene dimensión 4.

La Figura 3.11 muestra el resultado de aplicar Harris sobre una imagen de ejemplo. La textura presenta numerosas esquinas, que se marcan en verde en la imagen de la derecha.

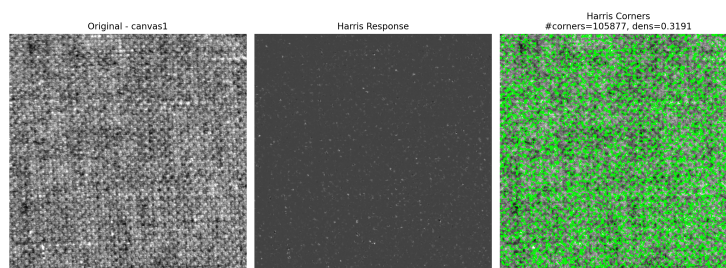


Figura 3.11: Harris

Clasificación de texturas

A continuación se muestran las métricas obtenidas empleando el descriptor basado en Harris. En las Figuras 3.12 y 3.13 se observa que algunas agrupaciones de

clases no son visualmente evidentes; esto puede justificar que la separación no sea tan trivial y que los modelos cometan ciertos errores.

No obstante, cabe destacar el reducido número de parámetros que necesitan los modelos para alcanzar una precisión muy elevada (Tabla 3.4, Figuras 3.12 y 3.13).

Tabla 3.4: Resultados de clasificación SGD(Harris)

Modelo	Descriptor	Num Params	Accuracy	Precision	Recall	F1-score
SGD	Harris	4	0.96	0.96	0.96	0.94
KNN	Harris	4	0.96	0.96	0.96	0.96

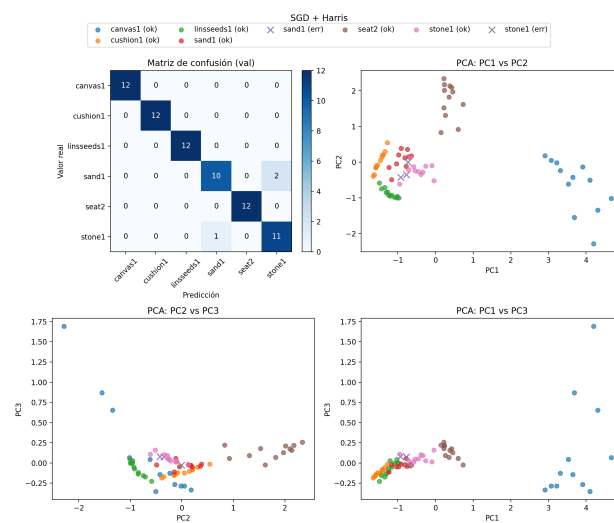


Figura 3.12: Matriz de confusión, SGD(Harris)

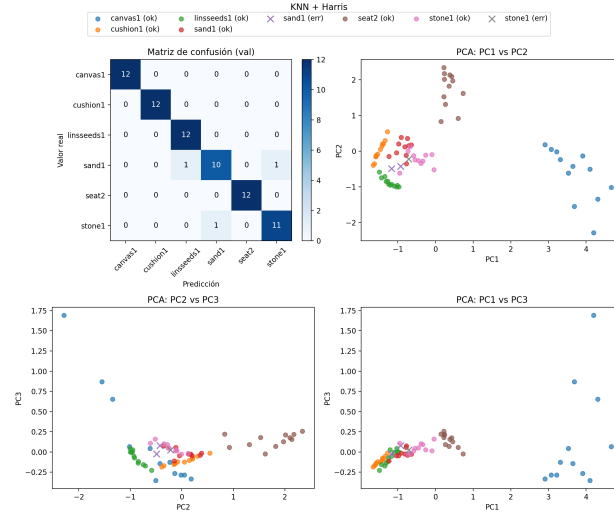


Figura 3.13: Matriz de confusión, KNN(Harris)

3.4.2. Canny Edge Detector

Al igual que Harris, Canny es un algoritmo de detección de bordes en imágenes. El procedimiento estándar sigue los pasos:

1. **Reducción de ruido:** se aplica un filtro gaussiano para suavizar la imagen y reducir el ruido.
2. **Cálculo de gradientes:** se calculan los gradientes en las direcciones x e y , G_x y G_y , y a partir de ellos la magnitud y la dirección. La dirección del gradiente es perpendicular al borde.
3. **Non-maximum suppression:** una vez obtenidas las magnitudes y direcciones, se aplica una supresión de no máximos para eliminar respuestas que no correspondan al borde más intenso en cada vecindario, obteniendo bordes más finos y definidos.
4. **Hysteresis thresholding:** se aplican dos umbrales, min_th y max_th . Si un píxel tiene una respuesta mayor que max_th , se considera borde; si es menor que min_th , se considera píxel plano. Si se encuentra entre ambos, se utiliza la histéresis: si está conectado a un píxel clasificado como borde fuerte (mayor que max_th), se mantiene como borde; en caso contrario, se descarta.

La Figura 3.14 ilustra el resultado de aplicar Canny. En general, este detector tiende a obtener más bordes que Harris.

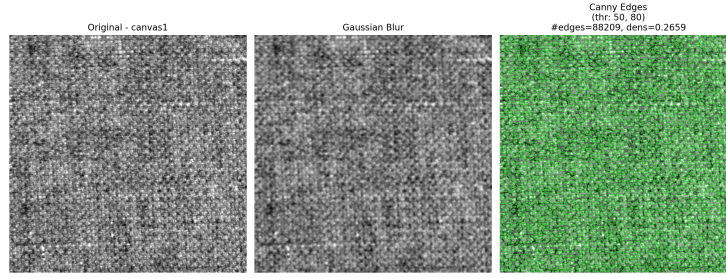


Figura 3.14: Canny

Clasificación de texturas

Siguiendo la misma idea que con Harris, en Canny también se resumen las respuestas en 4 parámetros que sirven de entrada a los modelos de ML. En este caso, las métricas se deterioran ligeramente en comparación con Harris; además, las nubes de puntos tras PCA muestran estructuras más complejas (por ejemplo, con forma de parábola), lo que sugiere una separación menos clara entre clases (Tabla 3.5, Figuras 3.15 y 3.16).

Tabla 3.5: Resultados de clasificación SGD(Canny)

Modelo	Descriptor	Num Params	Accuracy	Precision	Recall	F1-score
SGD	Canny	4	0.64	0.57	0.64	0.67
KNN	Canny	4	0.78	0.78	0.78	0.77

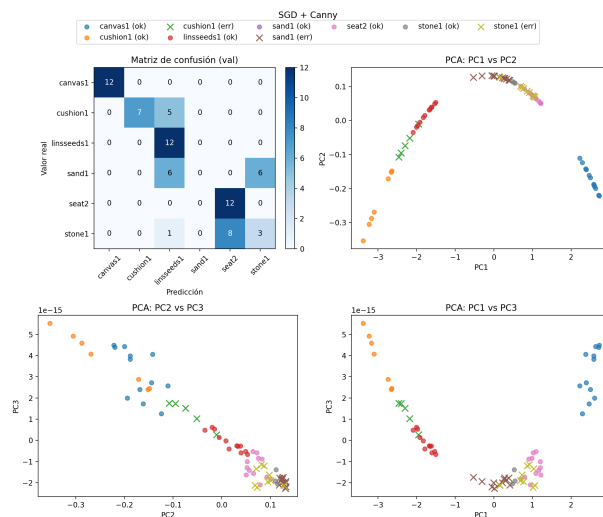


Figura 3.15: Matriz de confusión, SGD(Canny)

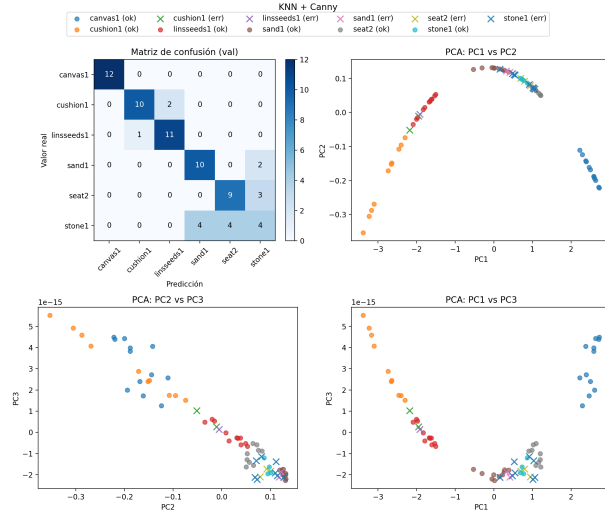


Figura 3.16: Matriz de confusión, KNN(Canny)

3.4.3. Scale Invariant Feature Transform (SIFT)

SIFT es un algoritmo de extracción de características locales que se centra en detectar puntos de interés en la imagen y describir su vecindario de forma robusta a cambios de escala, rotación e iluminación. Existe una variante denominada SURF que optimiza algunos aspectos de SIFT, pero no será estudiada en este trabajo.

El algoritmo SIFT consta de cuatro etapas principales:

1. **Scale-space extrema detection:** el objetivo es encontrar *keypoints* a distintas escalas.
 - a) Se utiliza *Laplacian of Gaussian* (LoG) con diferentes valores de σ para lograr invarianza a escala y detectar *blobs*, es decir, puntos de interés que se identifican como máximos o mínimos locales. Debido al coste computacional de LoG, se emplea una aproximación basada en *Difference of Gaussian* (DoG).
 - b) DoG consiste en calcular la diferencia entre dos imágenes suavizadas con Gauss con diferentes valores de σ .
 - c) La imagen se organiza en octavas, y a cada una se le aplican diferentes escalas de DoG. En el artículo original se emplean 4 octavas y 5 escalas por octava, con σ inicial de 1.6 y $K = \sqrt{2}$.
 - d) Se toma un píxel en una imagen DoG y se compara con sus 8 vecinos de la misma escala, los 9 vecinos de la escala inmediatamente inferior y los 9

de la escala inmediatamente superior. Si el píxel es mayor (o menor) que todos ellos, se considera un *keypoint*.

2. **Keypoint localization:** en esta etapa se refina la localización de los *keypoints* y se filtran aquellos poco estables. Algunos tendrán bajo contraste (aportan poca información) o estarán situados sobre bordes. Se ajusta la posición de los *keypoints* y se eliminan aquellos cuyo contraste no cumple un umbral. También se descartan los situados demasiado cerca de un borde, utilizando la matriz Hessiana H para evaluar la curvatura de la imagen y un cociente de autovalores que debe superar un umbral (típicamente 10).
3. **Orientation assignment:** se asigna una orientación dominante a cada *keypoint* para lograr invarianza a rotaciones. Para ello, se selecciona un vecindario alrededor del *keypoint*, se calcula el gradiente en cada píxel y se construye un histograma de 36 *bins* que cubren los 360 grados. El pico principal del histograma define la orientación del *keypoint*. Si existen picos adicionales con una magnitud superior al 80 % del máximo, se generan *keypoints* adicionales con esas orientaciones.
4. **Keypoint descriptor:** se busca describir el vecindario del *keypoint* de forma robusta a variaciones de iluminación y rotación. Se selecciona una ventana de 16×16 píxeles alrededor del *keypoint*, que se divide en 4×4 subregiones. En cada subregión se calcula un histograma de gradientes de 8 *bins*, que cubren los 360 grados. Cada subregión genera un vector de 8 componentes. Como hay 16 subregiones, el descriptor final tiene 128 componentes, resultado de concatenar los vectores de cada subregión.

Como resultado, para cada imagen se obtiene una matriz de dimensiones $N_{\text{keypoints}} \times 128$.

Utilizar directamente SIFT como entrada de un modelo de ML no es trivial, ya que cada imagen produce un número variable de *keypoints*, y por tanto descriptores de longitud distinta.

Una estrategia consiste en calcular estadísticos sobre el conjunto de descriptores de cada imagen (por ejemplo, media o desviación estándar) y seleccionar los K mejores *keypoints*, estandarizando así el tamaño del descriptor y convirtiéndolo en global. En esta sección se aplica precisamente esta variante como entrada para los modelos SGD y KNN.

Otra opción es utilizar técnicas de agregación como *Bag of Visual Words* (BoVW) o *Vector of Locally Aggregated Descriptors* (VLAD), que se estudiarán en secciones posteriores.

Clasificación de texturas

Las métricas obtenidas al aplicar SIFT, aplanar los descriptores y emplearlos como entrada para ambos modelos de clasificación son excelentes. Las separaciones visuales tras PCA son claras, lo que indica que los modelos han podido capturar fácilmente los patrones presentes en los datos (Tabla 3.6, Figuras 3.17 y 3.18).

Tabla 3.6: Resultados de clasificación SGD(SIFT)

Modelo	Descriptor	Num Params	Accuracy	Precision	Recall	F1-score
SGD	SIFT	256	1.00	1.00	1.00	1.00
KNN	SIFT	256	1.00	1.00	1.00	1.00

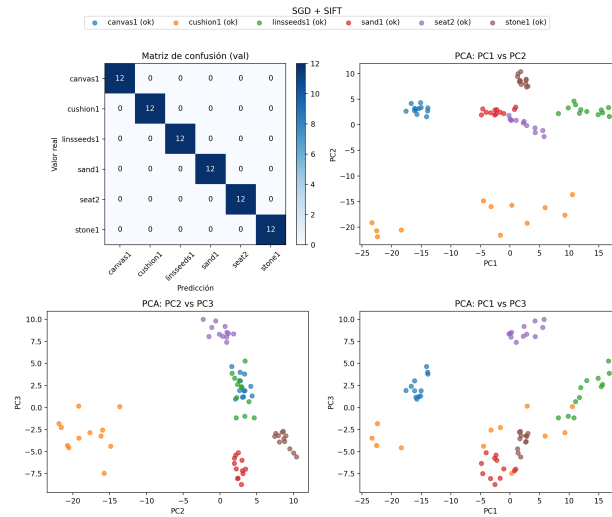


Figura 3.17: Matriz de confusión, SGD(SIFT)

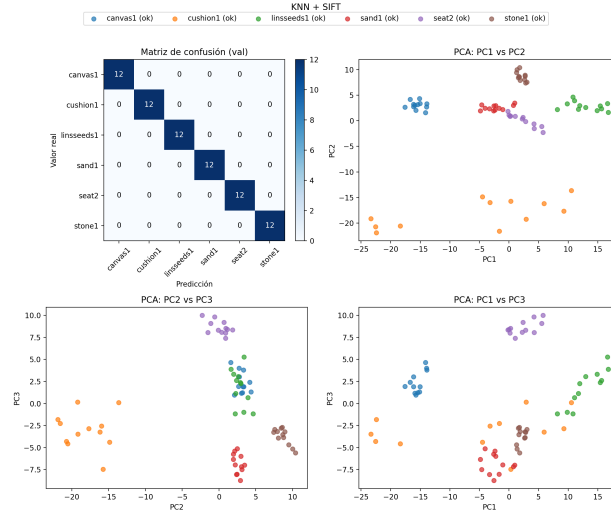


Figura 3.18: Matriz de confusión, KNN(SIFT)

3.4.4. Bag of Visual Words - BoVW

BoVW pretende resolver el problema de que los descriptores locales, como SIFT o SURF, generan un número variable de descriptores por imagen, lo que dificulta su uso directo en modelos de ML.

El proceso consta de tres etapas:

1. **Feature extraction:** se extraen descriptores SIFT de los *keypoints* de todas las imágenes de entrenamiento.
2. **Construcción del vocabulario:** a partir de todos los descriptores extraídos se agrupan los vectores en clústeres para formar el vocabulario o *codebook*. Para ello, se utiliza K-means.

Para construir el K-means hay que definir el número de clústeres, es decir, el tamaño del *codebook*. Típicamente se multiplica el número de clases por un factor, a menudo igual a 10.

3. **Representación de la imagen:** cada imagen se representa mediante un histograma de las palabras visuales que aparecen en ella. Para ello, se extraen los *keypoints* de la imagen, se asigna a cada uno la palabra visual más cercana del *codebook* y se cuenta cuántas veces aparece cada palabra. Este histograma se utiliza como descriptor global de la imagen.

El histograma generado para las imágenes de entrenamiento se emplea como vector de entrada para el clasificador. Para realizar inferencia sobre una imagen nueva,

se repite el proceso: extracción de *keypoints*, asignación al *codebook*, construcción del histograma y clasificación mediante el modelo entrenado.

La Figura 3.19 muestra un ejemplo de los histogramas obtenidos en el conjunto de entrenamiento tras aplicar BoVW y generar el *codebook*.

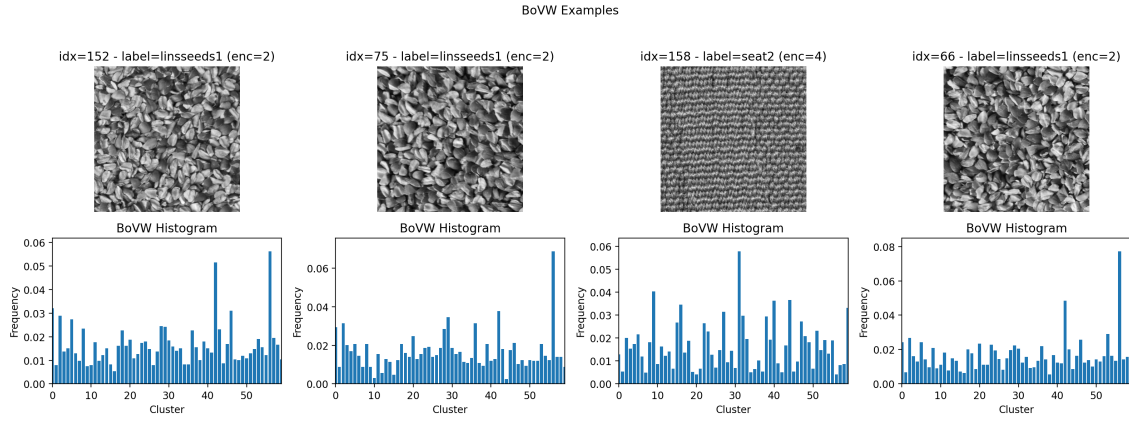


Figura 3.19: BoVW

Clasificación de texturas

Al igual que en los ejemplos anteriores, se entrenan y evalúan los modelos de ML (SGD y KNN). La diferencia aquí radica en que las características de entrada son los histogramas generados por BoVW.

La *performance* en este caso es muy buena: se obtienen métricas excelentes y una visualización con PCA que muestra una clara separación entre las distintas clases (Tabla 3.7, Figuras 3.20 y 3.21).

Tabla 3.7: Resultados de clasificación SGD(SIFT + BoVW)

Modelo	Descriptor	Num Params	Accuracy	Precision	Recall	F1-score
SGD	SIFT + BoVW	60	1.00	1.00	1.00	1.00
KNN	SIFT + BoVW	60	1.00	1.00	1.00	1.00

3.4.5. Vector of Locally Aggregated Descriptors - VLAD

VLAD, al igual que BoVW, busca resolver el problema de la dimensión variable de los descriptores locales. Sin embargo, en lugar de contar ocurrencias de palabras visuales, agrega los residuos entre descriptores y centros del *codebook*, capturando más información sobre la distribución de los descriptores.

Los pasos son similares a BoVW, con las siguientes particularidades:

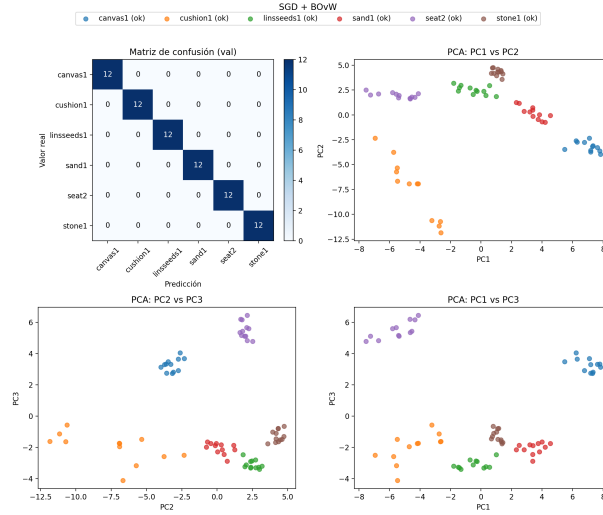


Figura 3.20: Matriz de confusión, SGD(SIFT + BoVW)

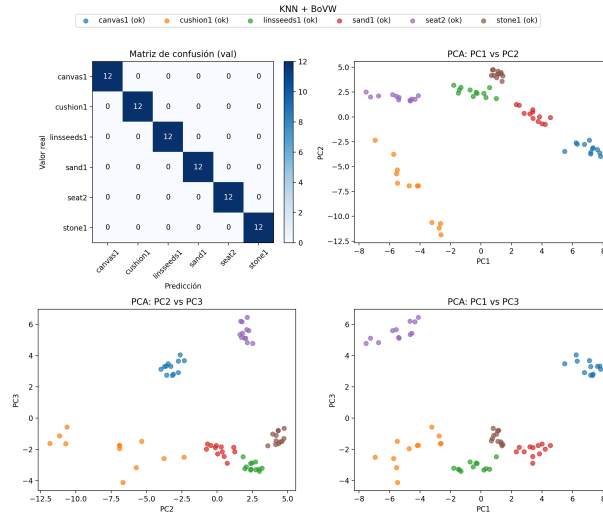


Figura 3.21: Matriz de confusión, KNN(SIFT + BoVW)

1. **Feature extraction:** se extraen descriptores locales (en este caso SIFT) sobre el conjunto de imágenes de entrenamiento.
2. **Codebook creation:** se utiliza K-means, del mismo modo que en BoVW, para construir el *codebook*.
3. **Residual encoding:** para cada descriptor local de una imagen se busca el centro del *codebook* más cercano y se calcula el vector residual como la diferencia entre el descriptor y dicho centro. Se suman todos los residuos asociados a cada centro, generando así un vector por centro.

4. **VLAD vector construction:** se concatenan todos los vectores residuales asociados a cada centro del *codebook*, formando el descriptor VLAD de la imagen.
5. **Vector normalization:** finalmente, se normaliza el descriptor (por ejemplo, con L2) para hacerlo más robusto.

Clasificación de texturas

Al igual que con BoVW, en VLAD se emplea el mismo número de parámetros y el rendimiento es excelente, con una separación clara de las clases en las representaciones PCA (Tabla 3.8, Figuras 3.22 y 3.23).

Tabla 3.8: Resultados de clasificación SGD(VLAD)

Modelo	Descriptor	Num Params	Accuracy	Precision	Recall	F1-score
SGD	VLAD	60	1.00	1.00	1.00	1.00
KNN	VLAD	60	1.00	1.00	1.00	1.00

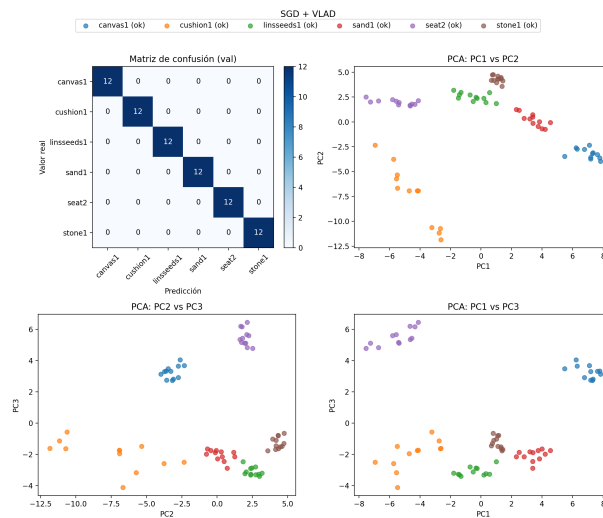


Figura 3.22: Matriz de confusión, SGD(VLAD)

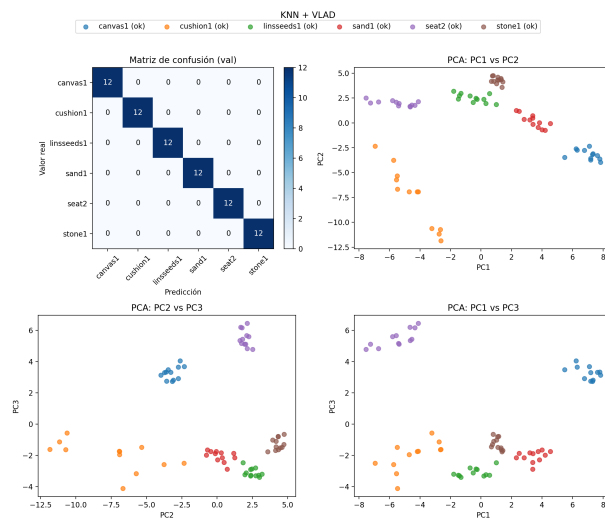


Figura 3.23: Matriz de confusión, KNN(VLAD)

Capítulo 4

Clasificación del dataset empleando Redes Neuronales

Otra de las estrategias para abordar problemas de visión por computador consiste en emplear redes neuronales para tareas como clasificación o detección. Para ello, se pueden utilizar componentes como las **FC** (*Fully Connected*), que son capas de neuronas completamente conectadas entre sí, y las **CNN** (*Convolutional Neural Networks*), también conocidas como redes convolucionales.

Para problemas más complejos, existen arquitecturas de redes neuronales convolucionales que combinan múltiples capas CNN con capas FC de forma optimizada. Sin embargo, dado que en este proyecto se dispone únicamente de 240 imágenes, se opta por una solución más sencilla: por un lado, convertir las imágenes en vectores y emplear una FC para realizar las predicciones, y por otro, diseñar una arquitectura simple basada en una CNN con una FC al final para tomar decisiones.

El proceso de entrenamiento y evaluación de los datos será diferente al mostrado previamente. En este caso, se dividirá el conjunto de datos en un 80 % para entrenamiento, 10 % para validación y 10 % para test (es decir, 192, 24 y 24 imágenes respectivamente, distribuidas de forma balanceada). Esta división permite evaluar el modelo durante el entrenamiento y visualizar las curvas de *accuracy* y *loss* a lo largo de las épocas, comparando los datos de entrenamiento y validación. Los datos de test se utilizarán para evaluar el rendimiento final del modelo sobre datos no vistos, extrayendo métricas como accuracy, precision, recall, F1-score y la matriz de confusión.

4.1. Fully Connected

Las redes neuronales totalmente conectadas (*Fully Connected*) se caracterizan por tener capas en las que todas las neuronas de una capa están conectadas con todas las neuronas de la siguiente (Figura 4.1). La concatenación de varias capas FC recibe el nombre de MLP (*Multilayer Perceptron*). Es importante tener en cuenta cómo escalan este tipo de redes y el número de parámetros que se deben entrenar, ya que pueden crecer rápidamente.

Cada neurona contiene pesos que deben ser ajustados durante el entrenamiento. En arquitecturas completamente conectadas, el número de parámetros crece exponencialmente conforme aumentan las dimensiones de entrada y salida.

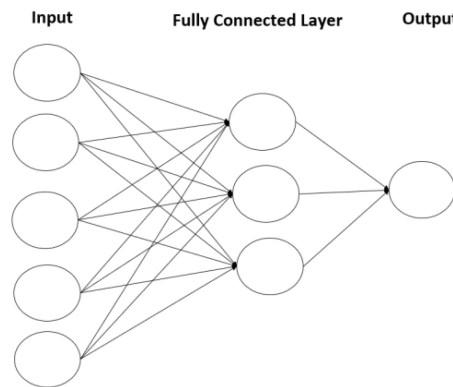


Figura 4.1: FC

En problemas de clasificación de imágenes, este enfoque no resulta eficiente, ya que la imagen debe ser aplanada a un vector. Para imágenes de resolución media, esto implica un gran número de parámetros a entrenar, lo cual puede volverse inabordable. Este problema se abordará con mayor detalle en la siguiente sección.

4.1.1. Clasificación de texturas

En esta clasificación, partiendo de imágenes de alta resolución (576x576 píxeles), se ha reducido la resolución a 256x256 píxeles para disminuir el número de parámetros sin perder información relevante. Estas imágenes contienen mucho "ruido", por lo que conservar características significativas es crucial.

Posteriormente, se normalizaron las imágenes para que sus valores estén entre 0 y 1. Esto permite que los pesos aprendidos por la red sean más pequeños y se mejore la eficiencia del entrenamiento. Las etiquetas del dataset fueron codificadas mediante *One Hot Encoding*.

La Figura 4.2 muestra la arquitectura utilizada en esta tarea.

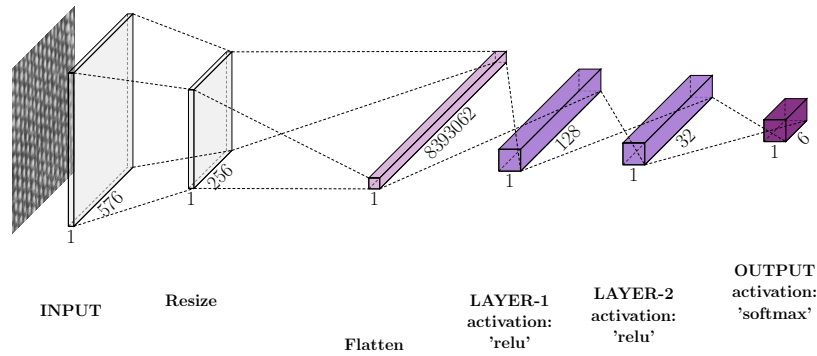


Figura 4.2: Arquitectura de FC

En la definición de esta arquitectura se integraron dos técnicas de regularización: **Dropout** con una tasa de 0.3 y regularización L2 con $\text{weight_decay} = 1e - 4$. Se utilizó el optimizador Adam con una tasa de aprendizaje (*learning rate*) de $8e - 4$, y como función de pérdida, la *categorical cross entropy*. Además, se implementó un mecanismo de *Early Stopping* que detiene el entrenamiento si la función de pérdida en validación no mejora en 15 épocas consecutivas.

La Figura 4.3 muestra la evolución del *accuracy* y *loss* durante el entrenamiento. El rendimiento obtenido es deficiente: no se observa progresión. La Figura 4.4 presenta la matriz de confusión del modelo sobre los datos de test, y la Tabla 4.1 resume el resto de métricas. El modelo tiende a predecir siempre la misma clase, lo que lo convierte en un clasificador ineficaz.

Tabla 4.1: Resultados de clasificación FC

Modelo	Num Trainable Params	Accuracy	Precision	Recall	F1-score
FC	8,393,062	0.17	0.30	0.17	0.05

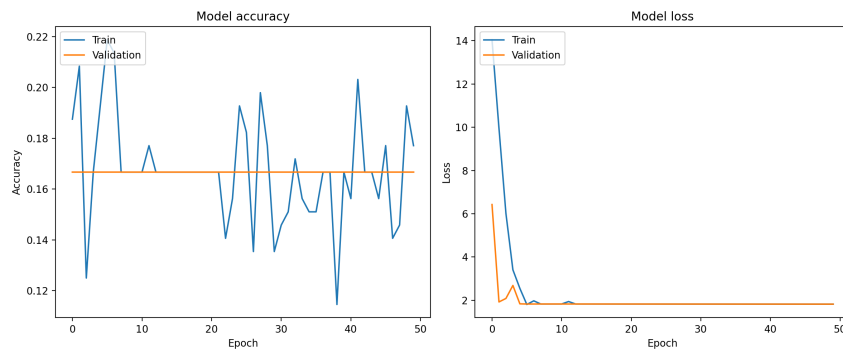


Figura 4.3: Evolución de *accuracy* y *loss* con arquitectura FC

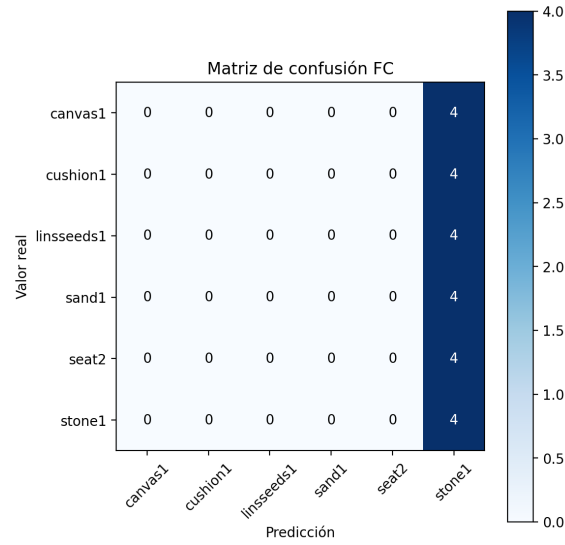


Figura 4.4: Matriz de confusión del modelo FC

4.2. CNN

Las *Convolutional Neural Networks* (CNN) representaron un gran avance en el campo de la visión por computador. Estas redes permiten construir arquitecturas más eficientes y con menos parámetros que las MLP tradicionales, además de aprovechar el paralelismo ofrecido por las GPUs para acelerar el entrenamiento.

Una arquitectura CNN sencilla para clasificación consta de varios bloques convolucionales encargados de extraer características. En estos bloques, se debe definir el número de filtros, el tamaño del *kernel*, el *stride* y la función de activación. Finalmente, se añaden capas FC con una función *softmax* para la clasificación.

4.2.1. Clasificación de texturas

Para abordar la clasificación del dataset de texturas, se diseñó una arquitectura adaptada a la tarea. Las imágenes originales de 576x576 píxeles fueron escaladas a 256x256 y normalizadas para que los valores de los píxeles estén en el rango $[0, 1]$.

La arquitectura definida consta de varios bloques CNN (convolución + función de activación + MaxPooling), seguidos de una capa FC final encargada de la clasificación. La arquitectura completa se muestra en la Figura 4.5.

Durante el entrenamiento se utilizó **Dropout** en la parte FC, con una tasa de 0.5, y se implementó también **Early Stopping**. En este caso, el rendimiento fue significativamente mejor que con la arquitectura FC. La Tabla 4.2 resume las métricas obtenidas sobre el conjunto de test, la Figura 4.6 muestra la evolución de la

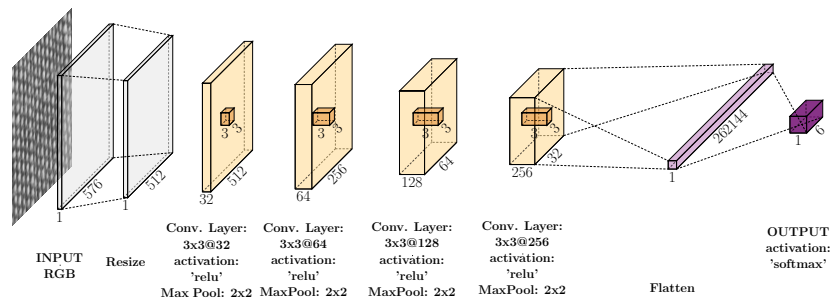


Figura 4.5: Arquitectura de CNN

función de pérdida y el *accuracy*, y la Figura 4.7 presenta la matriz de confusión correspondiente.

Tabla 4.2: Resultados de clasificación CNN

Modelo	Num Trainable Params	Accuracy	Precision	Recall	F1-score
CNN	8,777,350	0.96	0.97	0.96	0.96

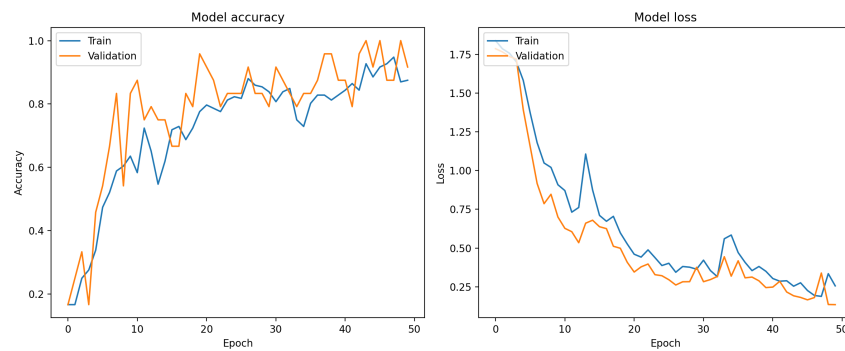


Figura 4.6: Evolución de *accuracy* y *loss* con arquitectura CNN

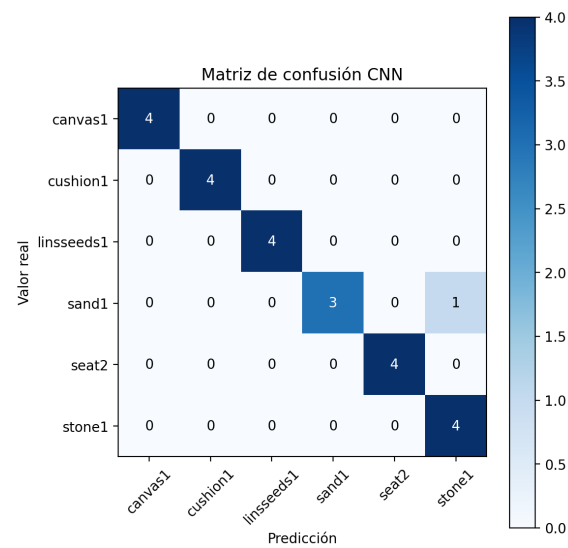


Figura 4.7: Matriz de confusión del modelo CNN

Capítulo 5

Conclusiones

En este trabajo se ha abordado el problema de clasificación de texturas del *Kyberg Texture Dataset* mediante dos enfoques complementarios: por un lado, el uso de descriptores clásicos combinados con modelos de *Machine Learning* tradicionales; por otro, el diseño y entrenamiento de arquitecturas de Redes Neuronales, tanto totalmente conectadas como convolucionales. A lo largo del documento se ha puesto de manifiesto cómo la elección adecuada de las características y del modelo de clasificación condiciona de forma crítica el rendimiento, pero también la complejidad computacional y la viabilidad de un futuro despliegue en producción.

En el ámbito de los descriptores clásicos (Capítulo 3), se ha evidenciado que no siempre un mayor número de parámetros implica un mejor rendimiento. HOG, a pesar de generar un vector de 181 476 componentes, ofrece resultados claramente inferiores a otros descriptores mucho más compactos: GLCM (24 componentes), LBP (32 componentes), Harris (4 componentes), SIFT aplanado (256 componentes) o técnicas de agregación como BoVW y VLAD (60 componentes) alcanzan precisiones prácticamente perfectas. Hay descriptores que no consiguen buenas performance como HOG o como en el caso de Canny y Harris que con la misma finalidad de detección de esquinas, a las que se les extrae ciertos estadísticos, se observa que Hanny es muy bueno mientras que Canny es más sensible y no consigue resultados tan buenos.

Si comparamos complejidad entre los diferentes descriptores empleados junto a sus clasificadores se seleccionaría aquellos modelos que tengan un menor número de parámetros de entrada y su complejidad sea menor, es decir que tenga un menor de componentes, ya que con BoVW y VLAD se añaden componentes como KMeans

para la creación de los diccionarios. En esta situación si tuviéramos que elegir, probablemente se escogería o GLCM o Harris.

En el Capítulo 4 se ha estudiado el uso de redes neuronales, comparando una arquitectura totalmente conectada (FC) con una CNN adaptada al problema. La red FC, a pesar de contar con más de 8 millones de parámetros entrenables, ha mostrado un rendimiento muy bajo, tendiendo a predecir siempre la misma clase y alcanzando una *accuracy* similar a la de un clasificador aleatorio. Otra de las desventajas encontradas es el alto número de parámetros que se tienen que entrenar, para que finalmente la clasificación sea tan baja. Mientras que con la arquitectura CNN, con un número de parámetros entrenables del mismo orden, se ha logrado una *accuracy* de alrededor del 96, %.. La CNN actúa como un extractor de características aprendidas, adaptadas de manera específica al problema, lo que la hace mucho más eficiente y robusta que la arquitectura FC.

La comparación entre los enfoques de descriptores clásicos + modelos de ML y las redes neuronales permite extraer varias conclusiones relevantes. En primer lugar, para este problema concreto de clasificación de texturas, no es imprescindible recurrir a redes neuronales para obtener clasificadores de alto rendimiento; de hecho, varios de los descriptores clásicos combinados con modelos sencillos (SGD o KNN) igualan o incluso superan el rendimiento de la CNN, con un coste computacional de entrenamiento e inferencia notablemente inferior y con una menor dependencia del tamaño del *dataset*. En segundo lugar, las redes neuronales, especialmente las CNN, siguen siendo una herramienta muy potente, capaz de aprender directamente representaciones discriminativas a partir de los datos sin necesidad de diseñar manualmente descriptores, además, por lo general, para dataset más complejos, las CNN necesitan un volumen de datos mucho más amplio. Por tanto, la elección entre un enfoque clásico o uno basado en redes neuronales debería, por tanto, depender del contexto, la cantidad de datos disponible, y las restricciones de tiempo de entrenamiento e inferencia.

Bibliografía

- [1] Kylberg texture dataset. <https://filedn.com/lkCRue0RhP07ercIrqFR12Y/datasets/KylbergTextureDatasetV1/>, 2024. Dataset.